Lab 4 Direct Map L1 Instruction Cache

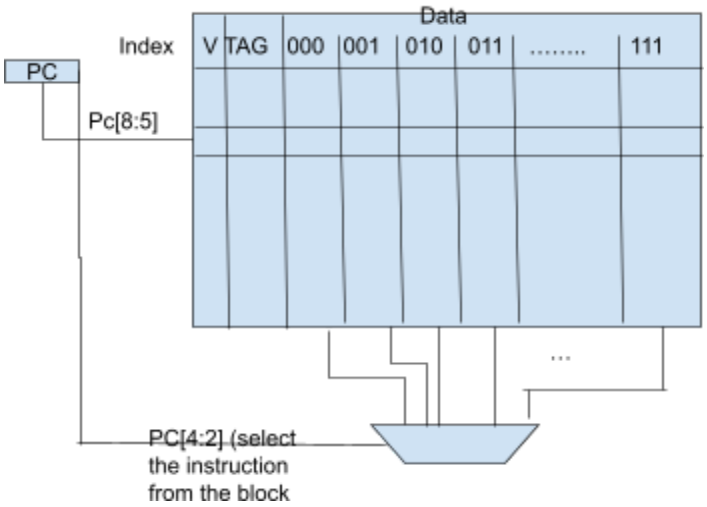
You will enhance your understanding of the Otter MCU in this lab by implementing a direct map L1 instruction cache. This cache, a crucial component in modern computer architecture, is designed to improve the efficiency of your pipeline implementation. If you still need to complete the pipeline lab, don't worry. You can still add the cache to the multicycle otter from 233 and experience its benefits.

A cache is a memory (storage accessible for reading and writing by the CPU) that stores locality in time(used recently) and space (instructions that are next to the ones used recently). It allows the CPU to act as if the memory storage capacity is vast and fast when, in reality, we have minimal fast storage, but we will fill it with this locality. When processors find the data in the cache, it is called a hit, else it is called a miss, and you will need to stall the PC for this lab for one clock cycle while the data is searched in the next level of memory (in this case de Instruction Memory) and the corresponding block is written into the cache.

For this lab, we will be working with a direct map L1 cache for Instruction Memory(IM). This cache, with its straightforward design of 16 blocks and 8 words per block, is a great starting point for understanding cache implementation. Since we are only allowed to read IM, there is no need for writing to instruction memory, and therefore, no need for using a dirty bit. There is also no LRU replacement for a direct map cache, making it even simpler to implement.

For a 32 bit PC:

TAG	Index	Word Offset	Byte Offset
31 ……..9	8 7 6 5	4 3 2	1 0



index=PC[8:5]

```
if(Valid[index] && TAG[index]==PC[31:9])
        //hit output the instruction 32 bits from multiplexer
        outInstruction=(IM[index]) selected from multiplexer with selector value equal to PC[4:2]
```

The template code (which you can change as much as you want) creates separate arrays for the tags, valid bits, and data, and each would be accessed based on the index of the PC. Since there are 16 blocks in the array the index was bits 8-5 of the PC. In addition, there are 8 words per block, so the specific word in the data array depends on bits 4-2 of the PC. The cache asynchronously checks whether there is a hit or a miss. If there is a hit, it outputs the correct instruction. Otherwise, if there is a miss, the PC will be stalled, so the cache outputs a no-op instruction (0x13) while a new block is fetched from instruction memory.

You probably need to create an FSM, to simplify the problem the template solution has an FSM included two states, ST_READ_CACHE and ST_READ_MEM. In the ST_READ_CACHE state, if there was a hit, the next state would be ST_READ_CACHE again since no updates had to be done to the cache. If there was a miss within ST_READ_CACHE, the FSM would signal a stall to the PC and the next state would be ST_READ_MEM where the PC is signaled to be stalled again.

TODO:

Implement and add this direct map cache L1 Instruction Memory to your pipelined or multicycle Otter.

I'm adding some starting code for the memory and the direct cache and the small FSM """BUT""" the code should be used as a starting code not as a complete solution. Modify the code so it works with your Otter and change it as much as you would like.

UPLOAD:

Please include your lab report with all team names, contributions, design decisions, and results. Test with the same test_all program as you did in Lab 3, and evaluate the performance difference between Labs 3 and 4.

```
module imem(
    input logic [31:0] a,
    output logic [31:0] w0,
    output logic [31:0] w1,
    output logic [31:0] w2,
    output logic [31:0] w3,
    output logic [31:0] w4,
    output logic [31:0] w5,
    output logic [31:0] w6,
    output logic [31:0] w7
    );
    logic [31:0] ram[0:16383];

    initial $readmemh("test_all.mem", ram, 0, 16383);
    //changed memory so it does output 8 words
    assign w0 = ram[a[31:2]];
     assign w1 = ram[a[31:2]+1];
    assign w2 = ram[a[31:2]+2];
    assign w3 = ram[a[31:2]+3];
    assign w4 = ram[a[31:2]+4];
    assign w5 = ram[a[31:2]+5];
    assign w6 = ram[a[31:2]+6];
    Assign w7 = ram[a[31:2]+7];
endmodule
```

```
module CacheFSM(input hit, input miss, input CLK, input RST, output logic update, output logic pc_stall);
  typedef enum{
      ST_READ_CACHE,
      ST_READ_MEM
 } state_type;

 state_type PS, NS;
 always_ff @(posedge CLK) begin
      if(RST == 1)
           PS <= ST_READ_MEM;
      else
           PS <= NS;
end

 always_comb begin
      update = 1'b1
      pc_stall = 0;
      case (PS)
           ST_READ_CACHE: begin
            update = 1'b0;
           if(hit) begin
               NS = ST_READ_CACHE;
           end
           else if(miss) begin
              pc_stall = 1'b1;
              NS = ST_READ_MEM;
             end

           else NS = ST_READ_CACHE;
         end

           ST_READ_MEM: begin
            pc_stall = 1'b1;
            NS = ST_READ_CACHE;
        end
      default: NS = ST_READ_CACHE;
      endcase
  end
endmodule
```

```
module Cache(
    input [31:0] PC,input CLK,input update,
    input logic [31:0] w0,input logic [31:0] w1,
    input logic [31:0] w2, input logic [31:0] w3,
    input logic [31:0] w4, input logic [31:0] w5,
    input logic [31:0] w6, input logic [31:0] w7,
    output logic [31:0] rd,output logic hit, output logic miss
);
parameter NUM_BLOCKS = 16;
parameter BLOCK_SIZE = 8;
parameter INDEX_SIZE = 4;
parameter WORD_OFFSET_SIZE = 3;
parameter BYTE_OFFSET = 0;
parameter TAG_SIZE = 32 - INDEX_SIZE - WORD_OFFSET_SIZE - BYTE_OFFSET;

logic [31:0] data[NUM_BLOCKS-1:0][BLOCK_SIZE-1:0];
logic [TAG_SIZE-1:0] tags[NUM_BLOCKS-1:0];
logic valid_bits[NUM_BLOCKS-1:0];
logic [3:0] index;

logic [TAG_SIZE-1:0]cache_tag, pc_tag;
logic [2:0] pc_offset;

initial begin
    int i;int j;
    for(i = 0; i < NUM_BLOCKS; i = i + 1) begin //initializing RAM to 0
            for(j=0; j < BLOCK_SIZE; j = j + 1)
              data[i][j] = 32'b0;
              tags[i] = 32'b0;
              valid_bits[i] = 1'b0;
    end
end

assign index = PC[8:5];
assign validity = valid_bits[index];
assign cache_tag = tags[index];
assign pc_offset = PC[4:2];
assign pc_tag = PC[31:9];

assign hit = (validity && (cache_tag == pc_tag));
assign miss = !hit;


always_comb begin
            rd = 32'h00000013; //nop
            if(hit) rd = data[index][pc_offset];
end

always_ff @(negedge CLK) begin
       if(update) begin
```

```
            data[index][0] <= w0;
            data[index][1] <= w1;
            data[index][2] <= w2;
          data[index][3] <= w3;
          data[index][4] <= w4;
          data[index][5] <= w5;
          data[index][6] <= w6;
          data[index][7] <= w7;
          valid_bits[index] <= 1'b1;

        end
end

endmodule
```